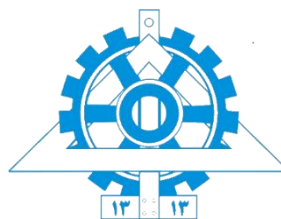




تمرین کامپیوتری شماره ۱



عنوان: شبیه سازی پروتکل های ارتباطی

درس: سامانه های بی درنگ نهفته

اساتید: دکتر محسن شکری ساز - دکتر مهدی کارگهی

رشته: مهندسی کامپیوتر

طراح تمرین: سهراب مرادی، هاتف رضایی، آرین فیروزی

نیم سال دوم سال تحصیلی ۱۴۰۴-۰۵

1	عنوان پروژه
1	۱-مقدمه
1	پیاده سازی پروتکل آدرس محور (I2C)
7	پیاده سازی پروتکل Full Duplex (USART)
10	پیاده سازی یک I2C Multiplexer
12	۳-جمع بندی و نکات پایانی

فهرست شکل‌ها

- 1 شکل 1 - دیاگرام اتصالات میکروکنترلر و سنسور برپایه I2C..... 1
- 2 شکل 2 - رابط GPIO سنسور VL530X..... 3
- 3 شکل 3 GPIO3 پورد ESP8266..... 3
- 4 شکل 4 - کلاس I2C در پورد ESP8266..... 3
- 5 شکل 5 - نمونه استفاده از پروتکل I2C در Sketch..... 4
- 6 شکل 6 - کلاس‌های پایه پروتکل I2C..... 4
- 7 شکل 7 - نحوه اتصال دو پورد به یک دیگر..... 5
- 8 شکل 8 - نمونه ای از یک Sketch..... 6
- 9 شکل 9 - زیر کلاس پروتکل USART..... 9
- 10 شکل 10 - کلاس پایه پروتکل USART..... 9
- 11 شکل 11 - دیاگرام اتصالات I2C Mux..... 10

عنوان پروژه

شبیه‌سازی پروتکل‌های ارتباطی

۱-مقدمه

در این تمرین قصد داریم با اضافه کردن پروتکل‌های ارتباطی به یک سیستم شبیه‌سازی، این شبیه‌ساز را دقیق‌تر و کامل‌تر کنیم. این تمرین شامل پیاده‌سازی سه بخش مختلف و پاسخ به تعدادی سوال از هر بخش است.

- پیاده‌سازی یک پروتکل آدرس محور (۱۲C)
- پیاده‌سازی یک پروتکل Full Duplex (USART)
- پیاده‌سازی یک ۱۲C Multiplexer

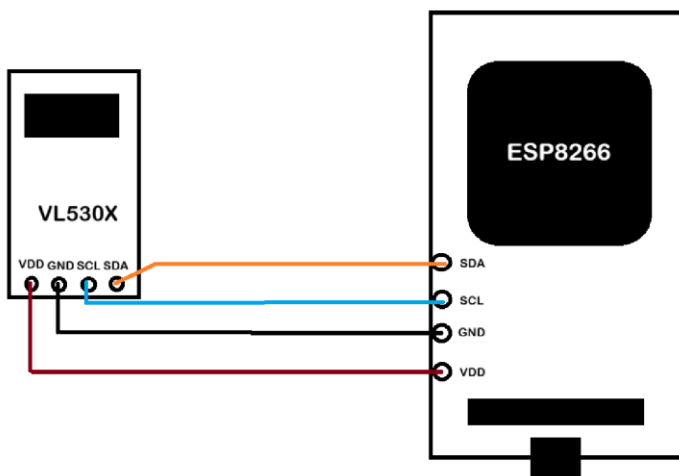
در ادامه به شرح هر بخش و ویژگی‌های مورد انتظار از آن می‌پردازیم.
نکته: پیش از پیاده‌سازی هریک از موارد خواسته شده در تمرین، حتماً راهنمای کد نویسی هر بخش را مطالعه کنید.

پیاده‌سازی پروتکل آدرس محور (۱۲C)

مجموعه کدهایی که در اختیار شما قرار گرفته است، بستری جهت تست اتصال سخت‌افزارهای گوناگون به یکدیگر و شبیه‌سازی سامانه‌های امبدد را مهیا می‌کند. اتصال دو یا چند ماژول فیزیکی به یکدیگر، علاوه بر الزامات سخت‌افزاری، شامل مجموعه الزامات نرم‌افزاری و قراردادی (پروتکل) نیز می‌شود. اکنون و در ابتدای این بخش از تمرین، امکان اتصال ماژول‌های سخت‌افزاری مختلف از منظر شبیه‌سازی فیزیکی وجود دارد و هدف این بخش از تمرین، افزودن پشتیبانی از پروتکل به آن است. پروتکل مد نظر نسخه بسیار ساده شده پروتکل ۱۲C است. از جهت الزامات سخت‌افزاری، این پروتکل به چهار پایه (PIN) روی برد سنسور و میکروکنترلر نیاز دارد، این چهار پایه شامل:

VDD: منبع تغذیه GND: زمین مشترک SCL: باس کلاک SDA: باس داده

هستند که باید نظیر به نظیر بین سنسور و میکروکنترلر به یکدیگر متصل شوند.



شکل ۱ - دیاگرام اتصالات میکروکنترلر و سنسور برپایه I2C

نحوه کار پروتکل مورد بحث به شرح زیر است:

- ۱- میکروکنترلر در ابتدا آدرس سنسور مقصد را روی لینک SDA می نویسد (در این پروتکل هر سنسور آدرس مخصوص به خودش را دارد که مقداری ثابت و از پیش تعیین شده است، به عنوان نمونه، آدرس سنسور VL530X برابر با مقدار ۰x۲۹ است (HEX))
 - ۲- سنسور پس از دیدن آدرس خودش، یک بیت ACK معادل ۱ روی SDA می نویسد و از این لحظه به بعد می تواند بر روی لینک SDA داده های دریافتی از محیط را بنویسد.
 - ۳- میکروکنترلر پس از دیدن بیت ACK آماده خواندن داده از سنسور است.
- سنسور و میکروکنترلر با استفاده از پایه SCL با یکدیگر همگام هستند بنابراین تداخلی در خواندن و نوشتن پیش نخواهد آمد. برای ساده سازی پایه SCL به صورت خودکار به کلاک میکروکنترلر متصل می شود، به همین دلیل شما کنترلی بر موج کلاک منتشر شده روی لینک SCL نخواهید داشت.
- برای پیاده سازی این بخش از تمرین شما باید یک میکروکنترلر و یک سنسور را به یکدیگر متصل کرده، بر روی این دو برد پروتکل I2C را پیاده سازی کنید و سپس با استفاده از آن، داده هایی را بین این دو قطعه منتقل کنید. نوع داده های انتقالی در انتهای راهنمای کدنویسی این بخش ذکر شده است.

سوالات:

- ۱) با توجه به اینکه I2C (نسخه ی اصلی که در درس با آن آشنا شده اید) یک پروتکل باس است در صورت اتصال بیشتر از یک سنسور با آدرس های مشابه چه اشکالی پیش می آید؟
- ۲) چهار مورد از اشکالات کلی این پروتکل ساده سازی شده نسبت به آنچه از I2C در درس آموخته اید بیان کنید.
- ۳) کارکرد کلاس های ByteVector, ByteStream و توابع getByte را توضیح دهید.

راهنمای کدنویسی:

در مسیرهای زیر یک برود ESP8266 و سنسور VL530X وجود دارد. هر برود یا سنسور شامل یک رابط GPIO است که برای این دو برود به شرح زیر است:

ESP8266: "{PROJECT_DIR}\include\CPS۴۰۴۲\Hardwares\Boards\Esp8266.h"

VL530X: "{PROJECT_DIR}\include\CPS۴۰۴۲\Hardwares\Sensors\VL530X.h"

```
template <BaudRate BR, BitRate BTR, typename WorkingVoltageTp>
requires std::is_base_of_v<AbstractVoltageLevel, WorkingVoltageTp>
struct Esp8266Gpio
{
public:
    Pins::Vdd<WorkingVoltageTp> vdd1 {BR, BTR, "Esp8266::vdd1"}; // Pin 0
    Pins::Gnd<WorkingVoltageTp> gnd1 {BR, BTR, "Esp8266::gnd1"}; // Pin 1

    Pins::Vdd<WorkingVoltageTp> vdd2 {BR, BTR, "Esp8266::vdd2"}; // Pin 2
    Pins::Gnd<WorkingVoltageTp> gnd2 {BR, BTR, "Esp8266::gnd2"}; // Pin 3

    Pins::Vdd<WorkingVoltageTp> vdd3 {BR, BTR, "Esp8266::vdd3"}; // Pin 4
    Pins::Gnd<WorkingVoltageTp> gnd3 {BR, BTR, "Esp8266::gnd3"}; // Pin 5

    Pins::Rx<WorkingVoltageTp> rx {BR, BTR, "Esp8266::rx"}; // Pin 6
    Pins::Tx<WorkingVoltageTp> tx {BR, BTR, "Esp8266::tx"}; // Pin 7

    Pins::Sda<WorkingVoltageTp> sda {BR, BTR, "Esp8266::sda"}; // Pin 8
    Pins::Scl<WorkingVoltageTp> scl {BR, BTR, "Esp8266::scl"}; // Pin 9

    Pins::Digital<WorkingVoltageTp> d0 {BR, BTR, "Esp8266::d0"}; // Pin 10
    Pins::Digital<WorkingVoltageTp> d1 {BR, BTR, "Esp8266::d1"}; // Pin 11
    Pins::Digital<WorkingVoltageTp> d2 {BR, BTR, "Esp8266::d2"}; // Pin 12
    Pins::Digital<WorkingVoltageTp> d3 {BR, BTR, "Esp8266::d3"}; // Pin 13
    Pins::Digital<WorkingVoltageTp> d4 {BR, BTR, "Esp8266::d4"}; // Pin 14
    Pins::Digital<WorkingVoltageTp> d5 {BR, BTR, "Esp8266::d5"}; // Pin 15
    Pins::Analog<WorkingVoltageTp> a0 {BR, BTR, "Esp8266::a1"}; // Pin 16
};
```

شکل GPIO3 برود ESP8266

```
template <std::uint64_t bar, std::uint64_t btr, typename WorkingVoltageTp>
requires std::is_base_of_v<AbstractVoltageLevel, WorkingVoltageTp>
struct VL530xGpio
{
public:
    Pins::Vdd<WorkingVoltageTp> vdd {bar, btr, "VL530x::vdd"}; // Pin 0
    Pins::Gnd<WorkingVoltageTp> gnd {bar, btr, "VL530x::gnd"}; // Pin 1
    Pins::Sda<WorkingVoltageTp> sda {bar, btr, "VL530x::sda"}; // Pin 2
    Pins::Scl<WorkingVoltageTp> scl {bar, btr, "VL530x::scl"}; // Pin 3
};
```

شکل 2 - رابط GPIO سنسور VL530X

همانطور که مشاهده می‌شود، رابط‌های GPIO شامل تعدادی پایه با کاربردهای متفاوت هستند، این پایه‌ها جهت اتصال بین ماژول‌های سخت‌افزاری مختلف مورد استفاده قرار می‌گیرند. وجود پین‌های خاص پشتیبانی از پروتکل‌های مربوطه را نمایان می‌سازد. به عنوان مثال، حضور دو پایه SDA, SCL در این دو برود، نشان می‌دهد که این دو باید از طریق پروتکل I2C به یکدیگر متصل شوند بنابراین پشتیبانی از این پروتکل باید به این دو برد اضافه گردد. در ادامه کد، مشاهده می‌کنید که هر دو کلاس دارای یک زیر کلاس به نام I2C هستند.

```
class I2C : public Protocols::AbstractI2C<Esp8266, Gpio>
{
public:
    explicit I2C(Esp8266* b) :
        Protocols::AbstractI2C<Esp8266, Gpio> {b}
    {}

    void
    init(Byte address) override
    {}

    void
    write(Byte byte) override
    {}

    Byte
    read() override
    {
        return 0;
    }

    void
    run(Gpio* gpio) override
    {}

    mutable i2c {this};
};
```

شکل 4 - کلاس I2C در برود ESP8266

وظیفه شما در این بخش از تمرین، پیاده سازی پروتکل شبیه به I2C با توجه به نیازهای تمرین است. همانطور که مشاهده می کنید، این کلاس شامل ۴ تابع است که وظیفه هر یک در ادامه توضیح داده میشود:

- (۱) تابع run: این تابع به صورت خودکار در هر سیکل CPU اجرا می شود، در این تابع شما باید با بررسی های بیت، ذخیره، پی بردن به آدرس یا داده، کاربرد پشت صحنه پروتکل را پیاده سازی کنید.
- (۲) تابع init: این تابع باید در تابع setup (مربوط به sketch که در ادامه توضیح داده خواهد شد) اجرا شود، در این تابع شما آدرس سنسوری که قصد برقرار ارتباط با آن دارید را به پروتکل اعلام می کنید.
- (۳) تابع read و write: این دو تابع حکم API های پروتکل را دارند، در تابع loop (مربوط به sketch) شما باید این دو تابع را برای خواند و نوشتن داده استفاده کنید، از طریق دسترسی به Object پروتکل از تابع node مشابه قطعه کد زیر:

```
static ByteStream<std::uint16_t> distanceStream;

if(node()->i2c.isDataAvailable()) distanceStream << node()->i2c.read();

if(distanceStream.isReady())
|   std::cout << "new distance: " << distanceStream.take() << std::endl;
```

شکل ۵ - نمونه استفاده از پروتکل I2C در Sketch

- (۴) همچنین توابع و متغیرهای کمکی لازم نیز در کلاس های AbstractProtocol و Protocols::AbstractI2C وجود دارند که استفاده از آنها برای پیاده سازی پروتکل لازم است:

```
template <typename Gpio>
class AbstractProtocol
{
public:
    explicit AbstractProtocol() = default;
    virtual void run(Gpio& gpio) = 0;

    virtual void write(Byte byte) = 0;
    virtual Byte read() = 0;

    bool
    isDataAvailable() const
    {
        return static_cast<bool>(AbstractProtocol<Gpio>::m_buffer.size());
    };

protected:
    std::queue<Byte> m_buffer;
};

template <typename BoardType, typename Gpio>
class AbstractI2C : public AbstractProtocol<Gpio>
{
    using BoardPtr = std::remove_pointer_t<boost::remove_cv_ref_t<BoardType>>;

public:
    explicit AbstractI2C(BoardPtr board) :
        m_board(board)
    {}

    virtual void init(Byte address) = 0;

protected:
    BoardPtr m_board;
    bool m_started {false};
};
```

شکل ۶ - کلاس های پایه پروتکل I2C

در ادامه و به عنوان مثال در تابع main، شما باید یک instance از هر دو کلاس بسازید و پایه‌ها را نظیر به نظیر به وسیله لینک‌ها به یک دیگر متصل کنید:

```
std::int32_t
main()
{
    Boards::Esp8266 esp8266;
    Sensors::VL530x vl530x;

    auto linkRed = std::make_shared<Link>();
    auto linkBlack = std::make_shared<Link>();
    auto linkGreen = std::make_shared<Link>();
    auto linkYellow = std::make_shared<Link>();

    CPS_SET_OBJECT_NAME(esp8266);
    CPS_SET_OBJECT_NAME(vl530x);

    CPS_SET_OBJECT_NAME_PTR(linkRed);
    CPS_SET_OBJECT_NAME_PTR(linkBlack);
    CPS_SET_OBJECT_NAME_PTR(linkGreen);
    CPS_SET_OBJECT_NAME_PTR(linkYellow);

    esp8266.gpio().vdd1.attachLink(linkRed);
    esp8266.gpio().gnd1.attachLink(linkBlack);
    esp8266.gpio().scl.attachLink(linkGreen);
    esp8266.gpio().sda.attachLink(linkYellow);

    vl530x.gpio().vdd.attachLink(linkRed);
    vl530x.gpio().gnd.attachLink(linkBlack);
    vl530x.gpio().scl.attachLink(linkGreen);
    vl530x.gpio().sda.attachLink(linkYellow);

    MicroController micro(&esp8266);
    Sensor disSen(&vl530x);

    micro.start();
    disSen.start();

    return Application::exec();
}
```

شکل 7 - نحوه اتصال دو برد به یک دیگر

نکته مهم، استفاده از ماکروهای CPS_SET_OBJECT_NAME و CPS_SET_OBJECT_NAME_PTR برای تنظیم نام متغیر است. این کار دیباگ را ساده‌تر و لاگ‌های شما را خواناتر می‌سازد. همچنین حتما توجه داشته باشد که در انتهای تابع main حتما تابع exec را فراخوانی کنید.

این بخش از کد معادل اتصال دو برد به یک دیگر در دنیای واقعی است.

همانطور که مشاهده می‌کنید، دو instance از این دو برد، به دو کلاس MicroController و Sensor پاس داده شده‌اند. در ادامه به بررسی این دو خواهیم پرداخت. این دو کلاس از جنس Sketch هستند. لازم است تا شما در مسیر زیر، اقدام به ساخت Sketch هایی برای پروژه خود کنید. در فرآیند واقعی، به عنوان مثال برای برنامه ریزی بردهای آردوینو، ما کدهایمان را در فایل‌هایی با پسوند .ino مینویسیم که نام آن‌ها Sketch است. این فایل‌ها باید شامل دو تابع باشند، setup و loop که تابع setup فقط یک بار در ابتدای شروع به کار میکروکنترلر فراخوانی می‌شود و تابع loop همانطور که از نامش نیز پیداست، مکرر تکرار خواهد شد. برای شبیه‌سازی این قسمت نیز، ما کلاس‌هایی به نام Sketch را تدارک دیده‌ایم.

```
#include <CPS4042/Hardwares/Sensors/VL530X.h>
#include <CPS4042/Sketchs/AbstractSketch.h>
#include <boost/random/mercenne_twister.hpp>
#include <boost/random/uniform_int_distribution.hpp>

class Sensor : public AbstractSketch<Sensors::VL530x>
{
public:
    explicit Sensor(Sensors::VL530x* node) :
        AbstractSketch<Sensors::VL530x> {node}
    {}

    std::int32_t
    setup(Sensors::VL530x::Gpio& gpio) override
    {
        std::cout << "vl530x setup completed." << std::endl;
        return 0;
    }

    std::int32_t
    loop(Sensors::VL530x::Gpio& gpio) override
    {
        /// do something
        return 0;
    }
};
```

شکل 8 - نمونه ای از یک Sketch

کدهای مربوط به هر یک از بوردهای شما باید در این Sketch ها نوشته شوند. مسیر Sketch ها:

"{PROJECT_DIR}\include\CPS4042\Sketchs"

دو نمونه Sketch در این فولدر وجود دارد که می‌توانید از آن‌ها برای پیاده‌سازی بخش اول استفاده کنید.

نحوه کارکرد سنسور:

سنسوری که مینویسد باید عددی بین ۰ تا ۴۰۰۰ (به صورت رندوم) را به میکروکنترلر باز گرداند. همانطور که می‌دانید، این رنج از اعداد در ۲ بایت قابل ذخیره هستند، همچنین توجه داشته باشید که توابع پروتکل با واحد بایت کار می‌کنند، بنابراین شما برای انتقال مقدار مورد نظر باید دوبار، ابتدا بایت اول و سپس بایت دوم را ارسال کنید. همچنین برای اطمینان از عملکرد صحیح، بایت سوم نیز باید منتقل شود، بایت سوم حاصل تفریق بایت بزرگ‌تر عدد (از جهت اندازه، نه از جهت جایگاه در عدد دوبایتی اصلی) و بایت کوچک‌تر باشد (این مقدار همواره نامنفی است). در سمت میکروکنترلر، باید این بایت سوم به عنوان checksum بررسی شود، اگر checksum مقدار معتبری داشت، داده معتبر است و می‌توانید آن را به عنوان نتیجه چاپ کنید. برای پیاده‌سازی ساده‌تر این بخش، می‌تواند از کلاس ByteStream در فولدر Utils و همچنین کلاس ByteVector و تابع getByte در هدر Units/Byte استفاده کنید.

پیاده‌سازی پروتکل Full Duplex (USART)

در این قسمت به پیاده‌سازی یک نسخه ساده از پروتکل USART می‌پردازیم. این پروتکل برای ارتباط سریال دوطرفه (Full Duplex) استفاده می‌شود؛ یعنی دو دستگاه می‌توانند به طور همزمان داده ارسال و دریافت کنند. (برای ساده‌سازی Baud Rate را برابر کلاک مستر در نظر بگیرید)

در این ارتباط معمولاً یک دستگاه به عنوان میکروکنترلر یا برد اصلی و دستگاه دیگر به عنوان مازول یا سنسور در نظر گرفته می‌شود.

در پروتکل USART پایه لینک اصلی وجود دارد:

TX (Transmit) : خط ارسال داده

RX (Receive) : خط دریافت داده

VDD : تغذیه

GND : زمین مشترک

نکته مهم این است که کارکرد RX و TX در سنسور برعکس میکروکنترلر است که بالاتر توضیح داده شد، یعنی سنسور از TX می‌خواند و بر RX می‌نویسد.

در این حالت، هر دو طرف می‌توانند همزمان داده ارسال و دریافت کنند.

برای شبیه‌سازی این بخش لازم است یک هارد دیسک فرضی طراحی شود که از طریق USART با میکروکنترلر ارتباط برقرار کند. روند کار می‌تواند به این صورت باشد:

میکروکنترلر یک آدرس (بین ۰ تا ۲۵۵) را بر روی TX می‌نویسد. هارد دیسک آدرس را دریافت کرده و داده متناظر با این آدرس را از طریق RX به میکروکنترلر بازمی‌گرداند.

Sketch مربوط به هارد دیسک به صورت نیمه پیاده‌سازی شده در فولدر Sketchs موجود است.

به دلیل Full Duplex بودن این پروتکل، ارسال و دریافت می‌تواند به طور همزمان انجام شود و نیازی به توقف ارتباط برای تغییر جهت داده وجود ندارد.

در پروتکل USART داده به صورت فریم ارسال می‌شود. هر فریم شامل چند بخش است که ترتیب آن‌ها به شکل زیر است:

- Start Bit
- Data Bits
- Parity Bit
- Stop Bit

نحوه‌ی ارسال یک فریم در USART

Start Bit ○

لینک در حالت عادی روی مقدار ۱ (High) قرار دارد (این بخش به صورت پیش فرض توسط پردازنده‌ای که نوشته ایم مدیریت می‌شود). وقتی فرستنده می‌خواهد ارسال را شروع کند، خط را به ۰ (Low) می‌برد. این بیت به گیرنده اعلام می‌کند که یک فریم جدید در حال شروع است.

Data Bits ○

سپس بیت‌های داده ارسال می‌شوند. معمولاً ۸ بیت داده ارسال می‌شود (از بیت کم‌ارزش‌تر LSB شروع می‌شود، شما فقط داده را بوسیله ی تابع write روی پروتکل بنویسید، این بخش نیز توسط ما پیاده سازی شده است).

Parity Bit ○

برای تشخیص خطا استفاده می‌شود و می‌تواند Even یا Odd باشد. در بسیاری از ارتباط‌ها این بیت استفاده نمی‌شود. (در تمرین لازم به پیاده‌سازی این بیت نیست)

Stop Bit ○

در پایان، یک یا دو Stop Bit با مقدار ۱ (High) ارسال می‌شود تا پایان فریم مشخص شود و گیرنده برای فریم بعدی آماده شود. (این بخش نیز توسط ما پیاده‌سازی شده است، شما فقط لازم است تا به تابع hasBitToRead در تابع run مربوط به پروتکل گوش کنید)

مثال یک فریم USART

اگر بخواهیم داده‌ی ۰۱۰۱۰۱۰۱ را ارسال کنیم، فریم می‌تواند به شکل زیر باشد:

Start | Data | Stop

۰ | ۰۱۰۱۰۱۰۱ | ۱

یعنی ترتیب ارسال روی خط TX:

۰ ۰۱۰۱۰۱۰۱ ۱

سوالات:

۱) توضیح دهید چرا بیت های شروع و پایان با بیت های داده اشتباه گرفته نمی‌شود.

۲) باتوجه به نبود کلاک توضیح دهید این پروتکل چه طور داده‌ها را تشخیص می‌دهد.

۳) توضیح دهید آیا امکان پیاده‌سازی پروتکل USART به صورت نرم‌افزاری (با استفاده از پین های دیجیتال) در میکروکنترلرهایی که مجهز به سخت افزار مربوطه نیستند امکان پذیر است؟ (به عنوان مثال این امکان برای ۱۲C وجود دارد) دلایل خود را کامل شرح دهید.

راهنمای کدنویسی:

مانند بخش گذشته لازم است تا بورد مربوطه نوشته شود، اما این بار لازم است تا بورد از پروتکل USART پشتیبانی کند، بنابراین باید زیر کلاس زیر را داشته باشد:

```
class USART : public Protocols::AbstractUsart<Esp8266, Gpio>
{
public:
    explicit USART(Esp8266* b) :
        Protocols::AbstractUsart<Esp8266, Gpio> {b}
    {}

    void
    write(Byte byte) override
    {}

    Byte
    read() override
    {
        return 0;
    }

    void
    run(Gpio& gpio) override
    {}

} mutable usart {this};
```

شکل 9 - زیر کلاس پروتکل USART

```
template <typename BoardType, typename Gpio>
class AbstractUsart : public AbstractProtocol<Gpio>
{
    using BoardPtr = std::remove_pointer_t<boost::remove_cv_ref_t<BoardType>>>;

public:
    explicit AbstractUsart(BoardPtr board) :
        m_board(board)
    {}

protected:
    BoardPtr m_board;
    bool m_isFlagDetected {false};
    bool m_isWritingFinished {false};
};
```

شکل 10 - کلاس پایه پروتکل USART

توجه داشته باشید که فرکانس کاری بورد باید مستقل باشد و نباید آن را روی Driven قرار دهید. همچنین برای ساده‌سازی، BaudRate بورد را روی BaudRates::NotSpecified و BitRate را نیز روی BitRates::same(BaudRates::NotSpecified) تنظیم کنید. GPIO مورد نیاز را نیز برای این بورد بسازید. نام این بورد را Usb گذاشته و آن را در مسیر Hardwares/Comm بسازید. همچنین توجه داشته باشید که شما باید پروتکل پیاده‌سازی شده را بر روی پردازنده بورد install کنید، با نگاهی به constructor بورد VL530X می‌تواند نحوه این کار را ببینید. **توجه داشته باشید که پردازنده قابلیت مدیریت چندین پروتکل را به صورت همزمان دارد.**

نکته بسیار مهم: همانطور که گفته شد، در میکروکنترلر نوشتن روی Rx و در هارد دیسک نوشتن روی Tx نباید انجام شود! با مطالعه کلاس Transmitter و استفاده از توابع آن، از بروز خطا در منطق USART جلوگیری کنید.

هیچ گونه تغییر در کدهای کلاس Transmitter لازم نیست!

توجه داشته باشید که مانند بخش پیشین تابع main مربوطه و Sketch های لازم را پیاده‌سازی کنید!

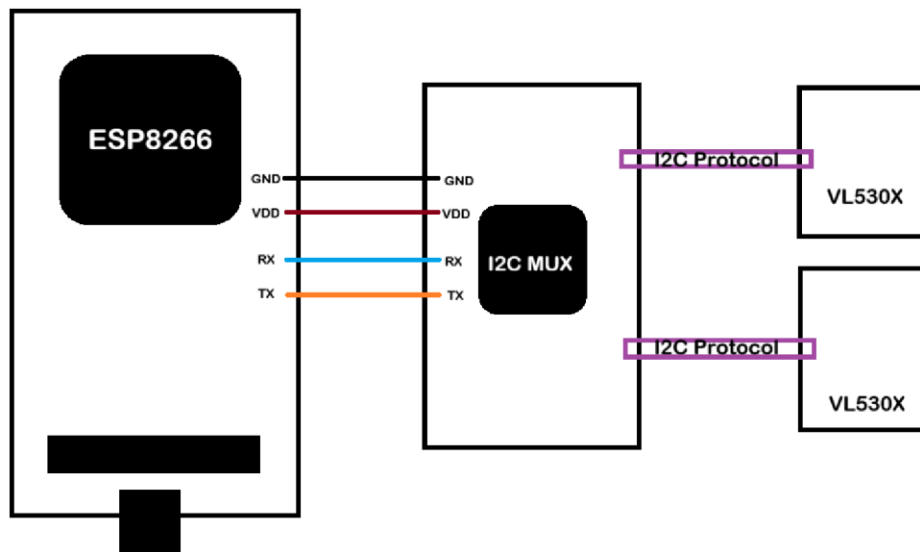
پیاده سازی یک I2C Multiplexer

در این قسمت چند سنسور که برپایه I2C کار می کنند باید به میکروکنترلر متصل شوند. از آنجا که اتصال سنسورهای I2C که آدرس یکسان دارند از طریق باس به میکروکنترلر (از طریق یک رابط I2C) ممکن نیست، از یک I2C Multiplexer استفاده می شود که به آن به اختصار I2C MUX گفته می شود.

در این ساختار، میکروکنترلر از طریق پروتکل USART با I2C Mux ارتباط برقرار می کند و Mux از طریق چند باس I2C جداگانه به سنسورها متصل می شود.

ساختار کلی ارتباط به صورت زیر است:

میکروکنترلر $\Rightarrow$ USART $\Rightarrow$ I2C MUX $\Rightarrow$ چند کانال I2C $\Rightarrow$ سنسورها



شکل 11 - دیاگرام اتصالات I2C Mux

نحوه عملکرد سیستم

۱. میکروکنترلر از طریق USART یک فرمان به Mux ارسال می کند که مشخص می کند کدام کانال I2C فعال شود. (شماره ی کانال I2C)
۲. Mux کانال مورد نظر را انتخاب می کند. (کانال = پروتکل های جداگانه I2C)
۳. سنسوری که روی آن کانال قرار دارد از طریق باس I2C با Mux ارتباط برقرار می کند.
۴. داده ی سنسور ابتدا به Mux ارسال می شود.
۵. Mux داده ی دریافت شده را از طریق USART به میکروکنترلر فورواردها می کند.

مزایای استفاده از I²C MUX

- امکان اتصال چند سنسور با آدرس یکسان
- جلوگیری از تداخل روی باس I²C
- مدیریت ساده‌تر چند سنسور توسط میکروکنترلر
- افزایش تعداد سنسورهای قابل اتصال به سیستم

به این ترتیب میکروکنترلر با انتخاب هر کانال می‌تواند به صورت جداگانه با هر سنسور ارتباط برقرار کند و داده‌های آن را دریافت نماید.

در این بخش از تمرین، شما باید حداقل ۲ سنسور I²C بسازید و در Sketch های مربوطه، داده‌های قابل تفکیک تولید کنید (به عنوان مثال یک سنسور اعداد بین ۰ تا ۲۰، سنسور دیگر اعداد بین ۵۰ تا ۱۰۰ و سنسور دیگر به ترتیب حروف یک نوشته را بازگرداند)، میکروکنترلر در هر سیکل، به وسیله ی یک شمارنده static به ترتیب داده یکی از سنسورها را از روی Mux بخواند و گزارش کند.

کلاس مربوط به I²C Mux را نیز در فولدر Hardware/Comm بسازید. توجه داشته باشید به تعداد کانال های موجود در این برد، باید Instance از کلاس I²C بسازید. نوشتن کلاسی عمومی برای I²C که بتوانید بجای ۳ بار بازنویسی کلاس، سه بار از آن Instance بگیرید در نمره دهی تاثیر مثبت دارد.

سوالات:

(۱) تابع reverse در هدر Byte.h و مکان‌های استفاده از آن را مطالعه کنید، کارکرد آن را توضیح دهید و بگویید آیا از همچنین عملیاتی در بوردهای واقعی استفاده می‌شود؟ راه حل دنیای واقعی آن چیست؟ (بیان عمیق مشکل در صورت عدم ارائه راه حل دنیای واقعی نیز قابل پذیرش است)

(۲) توابع takeNthBit در هدر Bit.h را توضیح دهید. تفاوت این دو در چیست؟

(۳) تابع attachPinToCommunicationClock از کلاس Board توضیح دهید. (از جهت کد و کاربرد)

۳- جمع بندی و نکات پایانی

- مهلت تحویل: ۱۴۰۵/۰۲/۱۱ ساعت ۲۳:۵۹
- پروژه در **گروه های ۴ نفره** انجام می شود.
- برای پیاده سازی این تمرین از **C++** استفاده کنید. کتابخانه boost، کامپایلر MinGW و کد شبیه ساز در اختیار شما قرار می گیرد. برای اجرای شبیه ساز به کامپایلری که از نسخه ۲۰ زبان C++ پشتیبانی می کند نیاز دارید، کامپایلر نسخه ویندوزی در اختیارتان قرار خواهد گرفت، اگر گروهی سیستم ویندوزی نداشتند از طریق ایمیل با مسئول دستیاران تماس برقرار کنند و مشخصات دقیق سیستم عامل خود را برای ایشان ارسال کنند.
- در پیاده سازی شبیه ساز از بخش های header-only کتابخانه boost استفاده شده است بنابراین نیازی به build کردن این کتابخانه ندارید، اما نسخه ی کامل build شده برای ویندوز در اختیارتان قرار خواهد گرفت (از header ها فارغ از سیستم عامل می توانید استفاده کنید)، لطفا include path ها را در فایل CMakeLists.txt به درستی تنظیم کنید.
- ساختار صحیح و تمیزی کد برنامه، **بخشی از نمره ی این پروژه شما خواهد بود**. بنابراین در طراحی ساختار برنامه دقت به خرج دهید.
- برای هر قسمت از کد، **گزارش** دقیق و شفاف بنویسید. کدهای ضمیمه شده بدون گزارش مربوطه نمره ای نخواهند داشت. در گزارش خود، نحوه شکست کار بین اعضای گروه را به صورت دقیق توضیح دهید، گزارش را می توانید در فایل Markdown (.md) یا word بنویسید.
- هدف این تمرین یادگیری شماسست. لطفا تمرین را خودتان انجام دهید و تجربیات خود در بخش محول شده را با هم گروهی های خود به اشتراک بگذارید.
- سؤالات خود را تا حد ممکن در **تالار پرسش و پاسخ** مطرح کنید تا سایر دانشجویان نیز از پاسخ آن بهره مند شوند. در صورتی که قصد مطرح کردن سؤال خاص تری دارید، از طریق ایمیل های زیر ارتباط برقرار کنید.

Arian.Firoozi@ut.ac.ir ○

Hatef.Rezaee@ut.ac.ir ○

M.Moradi1998@ut.ac.ir ○

موفق باشید.